Trabajo Práctico 2
Programación Lógica

Paradigmas ~~de Lenguajes~~ de Programación
2do cuatrimestre 2026

1. Introducción

Popong es un juego de tipo de rompecabezas lógico que consiste en vaciar una grilla eligiendo distintas posiciones sobre ella. Cada posición de la grilla puede estar libre u ocupada. Las posiciones ocupadas están pintadas de un cierto color. Cada jugada consiste en elegir una de las posiciones libres de la grilla. Cada vez que se elige una posición libre se liberan ciertas otras posiciones, dependiendo de la jugada. El juego finaliza cuando la grilla queda completamente vacía o cuando ya no se puede seguir jugando.

Para determinar qué posiciones se liberan en cada jugada, se tienen en cuenta las posiciones vecinas de la posición elegida de la siguiente manera:

- Las posiciones vecinas a una posición son las posiciones ocupadas **más cercanas** en línea recta hacia cada una de las cuatro direcciones.
- Si la posición elegida está sobre un borde, no habrá ninguna vecina hacia la dirección del borde. Tampoco si todas las posiciones en una dirección están libres.
- En caso de haber posiciones vecinas a la posición seleccionada, las mismas se pueden liberar unicamente si hay al menos dos de ellas que tengan el mismo color.
- Si entre las vecinas de una posición libre existen vecinas de algún color para el cual hay más vecinas de ese color que de los demás colores presentes entre las vecinas, se liberan todas las vecinas del color mayoritario. Las demás no. En caso de empate se liberan las vecinas de todos los colores.
- Si no se cumplen estas condiciones, entonces no se puede jugar esa posición.

En la imagen de la figura 1 se muestra el transcurso de un juego en un tablero de 3×4. El segundo tablero es el resultante de hacer una jugada en la posición (2,2) del primer tablero. El tercero es el resultante de hacer una jugada en la posición (2,1) del segundo. En el tercer tablero no hay jugadas posibles. Si la primera jugada hubiera sido en la posición (2,3), el juego se podría haber ganado jugando luego en la posición (2,1).



Figura 1: Ejemplos de tableros de popong

1.1. Objetivo

En este trabajo práctico vamos a implementar un programa que resuelva el juego en Prolog.
Se dispone de un `Makefile` que permite cargar el entorno y ejecutar los tests de los ejercicios.

- `make repl` inicia swipl con el archivo `tp2.pl` cargado.
- `make test` corre los tests de `tests.pl` usando plunit.
- `make watch` corre los tests automáticamente cada vez que se modifica algún archivo `.pl`. Requiere `fswatch` y `xargs`.

Al final del archivo `tp2.pl` se define el predicado `guiTablero` que permite graficar un tablero en la terminal, junto con algunos tableros de ejemplo. Pueden ser utilizados de la siguiente forma

```
?- tablero(1, T), guiTablero(T).
```

El último ejercicio es opcional.

2. Ejercicios

2.1. Matrices

Comenzamos trabajando con matrices, representadas como listas de listas.

Ejercicio 1: esMatriz/1

Implementar el predicado `esMatriz(+ListaDeListas)` que es verdadero cuando `ListaDeListas` es una matriz. Esto es, todas las listas tienen la misma longitud.

Ejercicio 2: tamaño/3

Implementar el predicado `tamaño(+Matriz, -Filas, -Columnas)` que instancia en `Filas` la cantidad de filas de `Matriz` y en `Columnas` su cantidad de columnas. Se presupone que `Matriz` es una matriz.

Ejercicio 3: coordenadaEnRango/2

Implementar el predicado `coordenadaEnRango(+Matriz, +Coordenada)` que es verdadero cuando la coordena `Coordenada` (representada a partir de un par de números correspondientes a la fila y la columna) pertenece a `Matriz`. Al estar hablando de matrices y no de posiciones de memoria, empezamos a contar desde 1 así que la primera fila es la número 1. Se presupone que `Matriz` es una matriz.

- ¿Es este predicado reversible en alguno de sus argumentos? Justifique.

2.2. Tableros

Ahora utilizaremos las matrices para representar tableros del juego. Un tablero se va a representar como una matriz de átomos, donde cada átomo puede ser `libre` para representar una posición libre del tablero o uno de los colores que instancia el predicado `color` (implementado en el archivo `popong.pl`).

Contamos con los siguientes predicados para observar tableros:

- `valorEnCoordenada(+Tablero, ?Coordenada, ?Valor)` que se puede usar de las siguientes maneras:
  - Si `Coordenada` está instanciada y `Valor` no, instancia en `Valor` el valor en las coordenadas `Coordenada` del tablero `Tablero`.
  - Si `Valor` está instanciada y `Coordenada` no, instancia en `Coordenada` todas las coordenadas del tablero `Tablero` que tienen el valor `Valor`.
  - Si ni `Coordenada` ni `Valor` están instanciadas, instancia en `Coordenada` todas las coordenadas del tablero `Tablero` y en `Valor` sus correspondientes valores.
- `coordenada(+Tablero, -Coordenada)` que instancia en `Coordenada` todas las coordenadas del tablero `Tablero`.
- `libre(+Tablero, ?Coordenada)` que instancia en `Coordenada` todas las coordenadas libres del tablero `Tablero` (o, si `Coordenada` ya está instanciada, verifica si la coordenada `Coordenada` está libre en `Tablero`).
- `color(+Tablero, +Color, ?Coordenada)` que instancia en `Coordenada` todas las coordenadas del tablero `Tablero` que tienen el color `Color` (o, si `Coordenada` ya está instanciada, verifica si la coordenada `Coordenada` tiene color `Color` en `Tablero`).

Ejercicio 4: tableroConPosiciónLiberada/3 y tableroConPosicionesLiberadas/3

a) Implementar el predicado `tableroConPosiciónLiberada(+Tablero, +Coordenada, -TableroLiberado)` que instancia en `TableroLiberado` el tablero resultante de liberar la posición en la coordenada `Coordenada` del tablero `Tablero`. Esto es, `TableroLiberado` debería ser exactamente igual a `Tablero` excepto en la posición `Coordenada` que debe estar libre. Se presupone que `Tablero` es una matriz, que la coordenada `Coordenada` está en el rango de `Tablero` y que la posición `Coordenada` de `Tablero` no está libre.

b) Implementar el predicado `tableroConPosicionesLiberadas(+Tablero, +Posiciones, -TableroLiberado)` que instancia en `TableroLiberado` el tablero resultante de liberar las posiciones `Posiciones` del tablero `Tablero`. Esto es, `TableroLiberado` debería ser exactamente igual a `Tablero` excepto en las posiciones `Posiciones` que deben estar libres. Se presupone que `Tablero` es una matriz y que todas las coordenadas de `Posiciones` están en el rango de `Tablero` y que esas posiciones de `Tablero` no están libres.

2.3. Vecindad

Necesitamos conocer, dada una posición libre de un tablero, cuáles son las coordenadas de las posiciones ocupadas que pertenecen a su vecindad.

Contamos con los siguientes predicados auxiliares:

- `dirección(?Dirección)` que instacia en `Dirección` todas las posibles direcciones (o, si `Dirección` ya está instanciada, verifica que `Dirección` sea una dirección).
- `direcciones(-Direcciones)` que instancia en `Direcciones` una lista con todas las direcciones.
- `vecinaEnDir(+Coordenada, ?Dirección, -Vecina)` que instancia en `Vecina` la coordenada vecina (a distancia 1) a `Coordenada` en la dirección `Dirección` (o, si `Dirección` no viene instanciada, instancia cada par dirección-vecina (a distancia 1) en `Dirección` y `Vecina`, respectivamente).
- `hayOcupadaAl(+Tablero, +Coordenada, +Dirección)` que es verdadero cuando hay alguna posición ocupada en el tablero `Tablero` que esté en línea recta hacia la dirección `Dirección` desde la coordenada `Coordenada`. Se presupone que `Tablero` es una matriz y que la coordenada `Coordenada` está en el rango de `Tablero`.
- `ocupadaEnLíneaRectaAl(+Tablero, +Coordenada, +Dirección, -Ocupada)` que instancia en `Ocupada` la coordenada de la primera posición ocupada de `Tablero` que está en línea recta hacia la dirección `Dirección` desde `Coordenada`. Se presupone que `Tablero` es una matriz, que la coordenada `Coordenada` está en el rango de `Tablero` y que hay al menos una posición ocupada en `Tablero` desde `Coordenada` hacia la dirección `Dirección`.
- `ocupadasEnLíneaRecta(+Tablero, +Coordenada, -PosicionesOcupadas)` que instancia en `PosicionesOcupadas` las coordenadas de las primeras posiciones ocupadas de `Tablero` en línea recta hacia cada dirección desde `Coordenada`. Se presupone que `Tablero` es una matriz y que la coordenada `Coordenada` está en el rango de `Tablero`. Notar que la lista instanciada no necesariamente tiene 4 elementos ya que podría no haber posiciones ocupadas hacia todas las direcciones.

2.4. Jugadas

Con lo desarrollado, ya podemos empezar a simular un juego. Para eso, contamos con el predicado auxiliar `coloresPredominantes(+Tablero, +Coordenada, -Colores, -Coordenadas)` (que hace uso del predicado `ocupadasEnLíneaRecta` implementado en el ejercicio anterior), que instancia en `Colores` la lista con los colores que son predominantes en el tablero `Tablero` desde la coordenada `Coordenada`.

También contamos con el predicado `jugarNaive(+TableroInicial, -TableroFinal)` que instancia en `TableroFinal` el tablero resultante de jugar de manera naive. Un juego naive consiste en jugar en cada turno la primera posición que se encuentra, hasta que no haya más jugadas posibles. Notar que si se le piden más resultados a este predicado termina instanciando todos los posibles juegos sobre el tablero (puede haber resultados repetidos porque varias secuencias de jugadas distintas pueden terminar con el mismo tablero final). Se presupone que `TableroInicial` es una matriz.

Ejercicio 5: jugarPosición/3 y jugarPosiciones/3

a) Implementar el predicado `jugarPosición(+TableroInicial, +Coordenada, -TableroResultante)` que instancia en `TableroResultante` el tablero resultante de hacer una jugada en la posición `Coordenada` del tablero `TableroInicial`. Se presupone que `TableroInicial` es una matriz, que la coordenada `Coordenada` está en el rango de `TableroInicial` y que la posición en dicha coordenada está libre en `TableroInicial`.

b) Implementar el predicado `jugarPosiciones(+TableroInicial, +Jugadas, -TableroResultante)` que instancia en `TableroResultante` el tablero resultante de hacer jugadas en cada una de las posiciones de `Jugadas` sobre el tablero `TableroInicial`, en ese orden. Se presupone que `TableroInicial` es una matriz y que todas las coordenadas en `Jugadas` están en el rango de `TableroInicial` y que sus respectivas posiciones en `TableroInicial` están libres al momento de jugar sobre ellas.

Ejercicio 6: jugadasNaive/2

Implementar el predicado `jugadasNaive(+Tablero, -Jugadas)` que instancia en `Jugadas` la lista de jugadas que realizaría un jugador naive. Nuevamente, notar que si se le piden más resultados a este predicado termina instanciando todas las posibles secuencias de jugadas sobre el tablero (en este caso no deberían instanciarse resultados repetidos). Se presupone que `Tablero` es una matriz.

Ejercicio 7: victoriaConJugadas/2, juegoNaiveVictorioso/2 y mejorJuegoNaive/2

a) Implementar el predicado `victoriaConJugadas(+Tablero, +Jugadas)` que es verdadero cuando la lista de jugadas en `Jugadas` logra la victoria en el tablero `Tablero`. Esto es, el tablero resultante de jugar las jugadas de Jugadas sobre el tablero `Tablero`, en ese orden, está vacío. Se presupone que `Tablero` es una matriz y que cada coordenada de `Jugadas` está en el rango de `Tablero`.

b) Implementar el predicado `juegoNaiveVictorioso(+Tablero, -Jugadas)` que instancia en `Jugadas` la lista de jugadas que realizaría un jugador naive y tras la cual alcanzaría la victoria. Nuevamente, notar que si se le piden más resultados a este predicado termina instanciando todas las posibles secuencias de jugadas victoriosas sobre el tablero (en este caso no deberían instanciarse resultados repetidos). Se presupone que `Tablero` es una matriz.

c) Implementar el predicado `mejorJuegoNaive(+Tablero, -Jugadas)` que instancia en `Jugadas` la lista de jugadas correspondiente a alguno de los mejores juegos victoriosos que realizaría un jugador naive (si hay más de uno, debería instanciarlos todos) sobre el tablero `Tablero`. Un juego es mejor que otro si requiere menos jugadas para terminar. Se presupone que `Tablero` es una matriz.

Ejercicio 8: tableroEnElQueNaiveGanaEmpezandoPor/2 (OPCIONAL)

Implementar el predicado `tableroEnElQueNaiveGanaEmpezandoPor(-Tablero, +Coordenada)` que instancia en `Tablero` cada tablero en el que un jugador naive podría llegar a salir victorioso si su primera jugada es en la posición de la coordenada `Coordenada`.

3. Pautas de Entrega

Importante: Se espera que la elaboración de este trabajo sea 100 % de los estudiantes del grupo que realiza la entrega. Así que, más allá de que pueden tomar información de lo visto en las clases o consultar información en la documentación de SWI-Prolog u otra disponible en Internet, no se podrán utilizar herramientas para generar parcial o totalmente en forma automática la resolución del TP (e.g., ChatGPT, Copilot, etc). En caso de detectarse esto, el trabajo será considerado como un plagio, por lo que será gestionado de la misma forma que se resuelven las copias en los parciales u otras instancias de evaluación.

El código debe poder ser ejecutado en SWI-Prolog. No es necesario entregar un informe sobre el trabajo, alcanza con que el código esté adecuadamente comentado. Pueden modificar el archivo `popong.pl` para hacer pruebas pero la entrega debería funcionar con el archivo original del esqueleto. Los objetivos a evaluar en la implementación de los predicados son:

- Corrección (el código debe cumplir con la funcionalidad pedida).
- Declaratividad (los nombres de variables, y predicados deberían reflejar el tipo y la intención).
- Legibilidad (el código debería ser fácil de leer y su estructura suficientemente simple como para poder comprenderlo sin mucho esfuerzo).
- Reutilización de predicados previamente definidos (es decir, evitar el código repetido).
- Uso de unificación, backtracking, generate and test y reversibilidad de los predicados que correspondan.
- Cada predicado auxiliar debe estar acompañado de su patrón de instanciación.
- No está permitido usar macros que colapsen cláusulas, como `;` y `->`.
- **Importante**: salvo donde se indique lo contrario, los predicados no deben instanciar soluciones repetidas. Vale aclarar que no es necesario filtrar las soluciones repetidas si la repetición proviene de las características de la entrada.

4. Referencias y sugerencias

Como referencia se recomienda la bibliografía incluída en el sitio de la materia (ver sección *Bibliografía* → *Programación Lógica*).

Se recomienda que, siempre que sea posible, utilicen los predicados ISO y los de SWI-Prolog ya disponibles. Recomendamos especialmente examinar los predicados y metapredicados que figuran en la sección *Útil* de la página de la materia. Pueden hallar la descripción de los mismos en la ayuda de `SWI-Prolog` (a la que acceden con el predicado `help`). También se puede acceder a la documentación online de SWI-Prolog.

En particular, seguramente les resulten útiles los siguientes predicados: `between/3`, `member/2`, `length/2`, `append/3`, `maplist/2`, `maplist/4`, `findall/3`, `not/1`.